

[[第一章微机系统论

微型计算机系统 (MCS) 包括硬件和软件；硬件包括微型计算机 (MC)、外围设备和电源；微型计算机 (MC) 包括微处理器 (MP)、存储器、I/O 设备和系统总线；微处理器 (MP) 包括算术逻辑单元 (ALU)、控制单元 (CU) 和寄存器阵列 (RA)；系统总线包括地址总线 (AB)、数据总线 (DB) 和控制总线 (CB) 系统总线：实现 CPU、I/O 设备、和存储器之间相互连接的总线。

[[第三章 8086/8088 微处理器及系统

内部结构组成：总线接口单元 (BIU) 和执行单元 (EU)

BIU：4 个 16 位的段寄存器 (CS、DS、SS、ES) 和 16 位指令指针 IP

EU：16 位 ALU、16 位标志寄存器 F、数据暂存寄存器、16 位通用寄存器 (数据寄存器：AX、BX、CX、DX；指针寄存器 SP、BP；变址寄存器 SI、DI)、EU 控电路**状态标志：**SF：结果的最高位 **OF：**有符号数溢出时为 1

CF：最高位产生进位或者借位时为 1 **PF：**结果低 8 位中含有偶数个 1 时为 1

AF：低四位向高四位进位或者借位时为 1 **ZF：**结果为 0 时为 1

DF：串操作方向标志，0 地址增加 1 减小 **IF：**中断标志位 IF=1 允许中断

总线周期：4 个周期组成，在 T3 后可插于 1 个或多个等待状态 TW,CPU 发出 READY 信号后进入 T4、T1 地址周期；T2 缓冲周期；T3 数据周期；T4 结束

规则字：存放字的低字节从偶地址开始存放，可一个周期完成存取 8086 中低位库（512K）与数据总线 D7-D0 相连，该库中每个地址为偶地址；高位库与 D15-D8 相连，该库中每个地址为奇地址。

字节访问周期：对字节的访问：需要一个总线周期；对低地址为偶地址的字的访问：需要一个总线周期。对低地址为奇地址字的访问：因为 A1 位不同，不能同时访问两片存储器，所以需要两个总线周期。 **最大(多处理器)、最小(单处理器)两种工作方式**

逻辑地址：段地址和偏移地址

物理地址 PA：段寄存器十六进制后添 0+偏移量 (EA)

有效地址 EA=基地址 (BX/BP) +变地址 (SI/DI) +位移量 D (0/8/16)

寻址方式

固定寻址：例：AAA 规定被调整的数总位于 AL 中

立即数寻址：MOV AL, 5；MOV AX, 3064H；MOVAX, 'AB'；送 BA 的 ASCII 码。注意字长一致 MOV AH, 3064H × (目标操作数不可以为立即数)

寄存器寻址 MOV AL, BH；MOV AX, BX；MOV AL, 64H；注意字长一致 MOV AH, BX ×

直接寻址：MOV AX, ES:[1680H]；隐含的段(没有标 ES 时)为数据段 DS；

变址(间接)寻址：MOV AX, [BX]，BX，数据段 DS；只要含有 BP，堆栈段 SS

MOV AX, [BX] PA=16d×DS+(BX)

MOV AX, ES:[BX] PA=16d×(ES)+(BX)

MOV AX, [BP] PA=16d×(SS)+(BP)

MOV AL, [BX+1000H] PA=16L×(DS)+(BX+1000H)

注意不允许使用 AX、CX、DX，只能是 BX 或者 BP如 MOV AX, [CX]×

变址(间接)寻址：偏移地址 = (SI)/(DI)+8 位/16 位偏移量

MOV AX, [DI] PA=16d×(DS)+(DI)

MOV AX, ES:[SI] PA=16d×(ES)+(SI)

注意：不允许使用 AX、CX、DX，只能是 SI 或者 DI 如 MOV AX, [AX]×

基址加变址(间接)寻址：必须用 BYTE PTR、WORD PTR 或 DWORDPTR 确定类型

MOV AX, [COUNT+BX+SI] PA=16d×(DS)+(0H+BX+SI)

串操作指令寻址、I/O 端口寻址：目标操作数不能为立即数或 CS

打*的指令：目标操作数不能为立即数+源和目标操作数不能同时为存储器操作数

MOV dst,src 传送指令：目标操作数不能为立即数或 CS；MOV 不影响标志位★

常见错误示例：MOV [SI], [BX]；错误，不能存储器之间直接传送，改为 BX

MOV [SI], IP ；错误，指令指针 IP 不能传送

MOV BX, AL ；错误，SRC 和 DST 的字长不一致

MOV CS, AX ；错误，目标操作数 dst 不能是 CS，也不能用立即数方式。

MOV ES, 2000H；错误，立即数不能直接送到段寄存器

MOV ES, DS ；错误目标操作数 dst 和源操作数 src 不允许同时为段寄存器

PUSH dst：入栈指令：(SP)←(SP)-2 ((SP)←(src) ★

POP dst：出栈指令：(dst)←(SP) (SP)←(SP)+2 ★

注意：PUSH 和 POP 不能使用立即数寻址方式；POP 指令的 dst 不能是 CS 寄存器；PUSH 和 POP 指令都不影响标志位。PUSH 和 POP 指令针对的是字，不是字节；

XCHG dst,src：交换指令

LEA reg,src: **有效地址送寄存器**执行操作:(reg)←offsetofsrc

ADD dst,src：加法指令；执行操作:(dst)←(src) + (dst)；影响状态标志位；★

ADD [2400H], 2AH 错误，没有指明类型，应该在[2400H]前说明是字、还是字节)

ADC src,src：带进位加指令；执行操作:(dst)←(src) + (dst) + CF；影响状态标志位★

INC opr：加 1 指令；执行操作:(opr)←(opr) + 1；★将目标操作数当作无符号数，完成加 1 操作后，结果仍保留在目标中。INC 指令不影响进位标志 CF，影响其它 5 位状态标志位。INC: INC [BX]是错误的，应为 INC BYTE PTR [BX]；

SUB dst,src：减法指令；执行操作:(dst)←(dst) - (src)影响状态标志位★

DEC opr：减 1 指令；执行操作:(opr)←(opr) - 1 影响状态标志位 (除了 CF) ★

NEG opr：求补指令；执行操作:(opr)← - (opr)，将目标操作数取负后送回目标(按位

取反后保持符号位不变加 1)

CMP opr1,opr2：比较指令；执行操作:(opr1) - (opr2)影响状态标志位★

目标操作数与原操作数相减，但不送回结果，只根据运算结果置标志位)

MUL src：无符号数乘法；字节操作(AX)←(AL)×(src)；字操作:(DX,AX)←(AX)×(src)

注意：AL(AX)为隐含的被乘数寄存器；AX(DX,AX)为隐含的乘积寄存器

IMUL src：带符号数乘法；执行操作：同 MUL；

区别 MUL 和 IMUL 指令，举例如下：假设 AL=FFH，BL=FFH

MUL BL 则有 AX=255×255=65025(FF01H)

IMUL BL 则有 AX= (-1) × (-1) =1

DIV src：无符号数除法；执行操作：

字节操作:(AL) ← (AX)/src 的商 (AH) ← (AX)/src 的余数

字操作:(AX) ← (DX,AX)/src 的商 (DX) ← (DX,AX)/src 的余数

IDIV src：带符号数除法；与 DIV 相同，但操作数必须是带符号数

移位指令和循环移位指令：备注：被移出的部分置 CF

SAL 算术左移：最低位用 0 来补充，最高为移入 CF

SAR 算术右移：高位用符号位左移 (符号位不变)，最低位用 0 补充

SHR 逻辑右移：SHL 逻辑左移、ROL 循环左移、ROR 循环右移、RCL 带进位循环左移、RCR 带进位循环右移。

AND dst,src；逻辑与；执行操作：(dst) ← (dst) (src) 有 0 则 0

OR dst,src；逻辑或；执行操作：(dst) ← (dst) (src) 有 1 则 1

NOT opr ；逻辑非；执行操作：(opr) ← (opr) 按位取反

XOR dst,src；异或 ；执行操作：(dst) ← (dst) (src) 不同为 1

TEST opr1,opr2；测试 (test) 执行操作：(opr1) (opr2)，与，设置标志，结果不送回**小结：**清零，用与运算；置 1，用或运算；取反，用异或运算（与 1 异或）

MOVSB,MOVSXB；字节传送、字传送。执行操作：[DS:SI] →[ES:DI]，将源串的值传送到相应的目标串单元中，同时修改指针 SI、DI。

SCAS(B/W 区分是字节或字)目标串：修改比较次，放在 AX 或 AL 中，需搜索串首元素的偏移地址在 DI 中，ZF=1 则找到；**LODS(B/W)**源串：源串首元素偏移地址在 SI 中，将其放入 AX/AL 中，SI 自动加减 1 (根据 DF) **STOS(B/W)**目标串：目标串首元素偏移地址在 DI 中，将 AX/AL 中内容写入 DI 所指目标串，DI 自动加减 1 (2)

压缩的 BCD 码：压缩的 BCD 码用 4 位二进制数表示一个十进制数位，整个十进制数用一串 BCD 码来表示。例如，十进制数 59 表示成压缩的 BCD 码为 0101 1001，十进制数 1946 表示成压缩的 BCD 码为 0001 1001 0100 1101。

非压缩的 BCD 码：非压缩的 BCD 码用 8 位二进制数表示一个十进制数位，其中低 4 位是 BCD 码，高 4 位是 0。十进制 78 表示成压缩的 BCD 码为 0000 0111 0000 1000。**ASCII 码：**要将 ASCII 码表示的数转换成 BCD 码很简单，只要把 ASCII 码的高 4 位清零即可。如 ASCII 码中，3 表示为 33H，将高位清零后就得到 BCD 码。

压缩的 BCD 码调整指令：执行加法指令 (ADD、ADC) 或减法指令 (SUB、SBB) 后，如果结果的低 4 位 (AL)D0-3 > 9 或 AF=1，则 (AL) ← (AL) +06H，AF 置 1；如果结果的高 4 位 (AL)D4-7 > 9 或 CF=1，则 (AL) ← (AL) +60H，且 CF 置 1。

处理器控制指令：

CLD：DF=0 正向地址递增**STD：**DF=1 **CLI：**禁止可屏蔽中断 IF=0**STI：**IF=1

CLC：CF 清零 **STC：**CF 置 1 **CMC：**CF 取反 **HLT** 暂停 **NOP** 空操作(3 周期)

指令名称	助记符	测试条件
无条件转移	JMP 目标号	
调用过程	CALL 过程名	
从过程返回	RET弹出值	
重复执行串指令	REP (CX)≠0	
	REPE / Z (CX)≠0且ZF=1	
	REPNE / NZ (CX)≠0且ZF=0	
循环控制指令	LOOP (CX)≠0	
	LOOPZ/E ZF=1且(CX)≠0	
	LOOPNZ/NE ZF=0且(CX)≠0	
带符号数		
	高于	JAE/JNB
	高于等于	JAE/JNB
	低于	JB/JNAE
	低于等于	JBE/JNA
	大于	JG/JNLE
	大于等于	JGE/JNL
	小于	JL/JNGE
	小于等于	JLE/JNG
	等于/结果为 0	JE/JZ ZF=1
	不等于/结果不为 0	JNE/JNZ ZF=0
	有进位/借位	JC CF=1
	无进位/错误	JNC CF=0

第四章 汇编语言程序设计

DD 5 DUP (? ,12,3 DUP (12,5)) ；占用 160 字节

标号

NEAR (近)：只能被标号所在段内的转移和调用指令访问（即段内转移）；

FAR (远) 可以被其他段的转移和调用指令访问（即段间转移），也可为段内转移。

ASSUME 段寄存器名:段名**例：**ASSUME CS:SEGA,DS:SEGB,SS:NOTHING (取消)

分析运算符：SEG:送段地址；**OFFSET:**送偏移值（或 LEA）；**TYPE:**送类型（DB 为 1）；**SIZE:**送数据总字节；**LENGTH:**送数据总项数；**HIGH:**送高 8 位；**LOW:**低八位

	FOO	2AH	ZERO
地址表达式： DB 字符串初始化只能用 DB	1	01H	ONE
DW 偏移地址；DD 段地址(高)+偏移地址(低)		00H	
例：	3	03H	TWO
FOO SEGMENT AT 55H ;段基址 55H		00H	
ZERO DB 2AH ;内容为 2AH		55H	
ONE DW ONE ; 内容为自己偏移值 0001H	7	09H	THRE
TWO DD TWO；自己段基址+偏移值 00550003H		00H	
THREE DD THREE+2 ;偏移值 7+2=00550009H		55H	
FOUR DD FOUR+2；内容为偏移值 B+2=000DH	B	ODH	FOUR
FIVE DW FIVE-ONE；内容为偏移值 D-1=000CH		00H	
SIX DB \$-ZERO ;内容为\$当前地址 F-0=0FH	D	OCH	FIVE
STRING1 DB 'AB' ;字符串(A 为低字节)		00H	
STRING2 DW 'AB' ; 字节(A 为高字节)	F	0FH	SIX
FOO ENDS		A	STR1
		B	
		B	STR2
		A	

程序的基本结构：顺序、分支、循环结构

顺序结构程序：没有分支、没有循环的直线运行程序

分支结构程序：程序的分叉一般用条件转移指令来产生

循环结构程序：程序中存在需要重复多次执行的程序段部分（循环体）。循环控制部分每循环一次检查循环结束条件，满足条件就停止循环，往下执行程序。

[[第六章输入输出与中断 信息可分为三类：数据信息、状态信息、控制信息

CPU 与外设之间数据传送的方式：程序传送方式（效率低）、中断传送方式（多个外设，效率高）、直接存储器存取 DMA 传送方式（快速传送大批量数据）

程序传送方式（用 IN 和 OUT 指令直接控制 I/O 和 CPU 之间数据交换的方式，包括无条件传送、条件传送）

无条件传送（同步传送）：针对规定的外设，对于少量数据最省时间，I/O 可以是简单数据锁存缓冲器、CPU 如同访问内存存储器一样访问这些外设接口，无需检测状态。

条件传送（异步传送）：在执行输入(IN 指令)或输出(OUT 指令)前，要先查询接口中状态寄存器的状态信息 READY。由该状态信息指示要输入的数据是否已“准备就绪”和输出设备是否“空闲”，状态信息只占 1 位数据，不同的外设可以共用一个端口的不同的位。**缺点：**CPU 必须作程序等待循环,不断测试外设的状态,直至外设为交换数据准备就绪时为止，大大降低了 CPU 的运行效率。

POLL: ; 查询式输入程序	POLL: ; 查询式输出程序
IN AL, S_PORT; 读端口信息	IN AL, S_PORT; 读端口信息
TEST AL, 80H; READY 在 D7 位	TEST AL, 80H; BUSY 在 D7 位
JE POLL ; 未 ready, 循环	JNZ POLL ; 正忙, 循环
MOV AL, DATA; ready, 读入数据	MOV AL, DATA; 空闲, 读取数据
OUT D_PORT, AL	OUT D_PORT, AL

中断传送：利用中断来实现 CPU 与外设之间的数据传送，就是中断传送方式。采用中断传送方式时，CPU 从启动外设到外设准备就绪这段时间，不像查询方式一样处于等待状态，而仅仅是在外设准备好数据传送的情况下才中止 CPU 执行的主程序,在一定程度上实现了主机和外设的并行工作，提高了 CPU 的工作效率。

DMA 传送：DMA 方式主要通过 DMA 控制器，让一些高速的 I/O 设备直接与内存高速交换数据，不必通过 CPU 也不需要软件介入。**DMA 传送过程：**①接口准备就绪发出 DMA 请求②CPU 通过 HOLD 接收 DMA 发出的总线请求③HLDA 送回总线允许信号使 DMA 接管总线控制权④DMA 把地址送地址总线⑤DMA 请求被接口数据⑥内存把数据送数据总线⑦接口锁存数据⑧DMA 传送结束，将 HOLD 变低电平，撤销总线请求，让出总线权⑨CPU 将 HLDA 变为低电平恢复对总线控制。它是总传输完单个数据或者数据块后才交出总线控制权。

三种数据传送方式的比较

程序控制方式：对于条件传送方式，CPU 常需要反复查询外设状态，只有当外设已准备好数据或不忙碌时，才能与 CPU 交换数据，采用该方式，CPU 的利用率很低。

中断方式：只有输入设备将数据准备好或者输出锁存器为定时才能向 CPU 发起中断请求。不需反复查询外设状态。微机中断技术简单灵活多用，CPU 利用率大大提高。

DMA 方式：由 DMA 控制器取代 CPU 接管总线，控制外设于内存间直接进行高速成批的数据交换，传输效率高，数据传输速度基本取决于外设和存储器的存取速度。

CPU 与外设之间的数据传送方式中，中断方式可用于解决 CPU 与低速外设之间的矛盾,提高 CPU 工作效率。**DMA 方式**适用于 CPU 与高速外设之间传送大批量的数据。

8086/8088 中断系统

中断系统具有以下功能：响应中断、处理中断、返回、实现优先级排队、高级中断源能中断低级的中断处理

中断源以及优先级原则：优先级高的中断源能够中断低级中断源的中断服务程序

	中断名	中断类型	说明
高	除法错	INT 0	商大于被除数
	用户定义	INT n	软
	INTO	INT 4	溢出
↑	NMI	INT 2	非屏蔽中断
	INTR	可屏蔽中断	由外设送入
	单步	INT 1	调试用

(INT 0-4 属于系统 (CPU) 自动产生)

中断源：输入、输出设备、数据通道、实时时钟、故障源、调试程序设置的中断源**中断向量表：**存放中断服务程序的入口地址。

共 256 类中断：每类中断的入口地址 (CS:IP) 占 4 个内存单元，共占用 1K 单元，一般置于存储器的最低端，位于 00000-003FFH 的范围内。

中断处理过程：中断请求 -> 中断响应 -> 中断处理 -> 中断返回，占用 2 个时钟

可屏蔽中断 (INTR) 过程：①中断申请 (INTR) ②中断响应 INTR③读取中断类型号④把 PSW、CS、IP 压入堆栈，保存断点⑤清除 IF 与 TF 标志⑥从中断向量表中取新的 CS、IP 值⑦转入中断服务程序⑧开放中断⑨IRET 使旧 IP、CS、PSW 从堆栈中弹出⑩返回被中断程序（下划线为主要操作）

可屏蔽中断 (INTR) 响应周期

1 执行第一个 INTA 周期：通知外设接收中断请求

2 执行第二个 INTA 周期：获得中断类型号

3 执行一总线写周期：PSW 状态字（标志寄存器）入栈

4 执行一总线写周期：CS 入栈

5 执行一总线写周期：IP 入栈

6 执行一总线读周期：将 4*2ⁿ+2 存储字中的值读入 Cs

7 执行一总线读周期：将 4*N 存储字中的值读入 IP 在执行中断响应程序前整个过程 CPU 实际执行了 7 个总线周期，对于非屏蔽与软中断跳过第 1，2 步，从第 3 步开始执行到第 7 步。

可屏蔽中断响应的条件 假设使用单片 8259A：中断触发条件，IRRi = 1；中断屏蔽触发器对应位 IMRi=0；中断请求源的本级或高级中断不处在服务状态 ISRj=0 (j 的优先级高于等于 i)；CPU 的中断允许标志 IF=1，可通过指令 STI 开中断，通过指令 CLI 关中断 (IF=0)；当前指令执行完毕(指令是程序的最小单位，不可被中断和拆散，)。以上条件缺一不可，任何一个条件不满足，中断请求都将被挂起。

常用 DOS 中断调用

中断向量：中断例程序的入口地址，存放于**中断向量表**。每个中断例程序的入口地址占 4 个字节，前 2 个存放中断入口的偏移地址 (IP)，后 2 个存放段地址 (CS)，共 256 个中断，从 00000H-003FFH,占用了 1024 个字节。

1.返回 DOS (矢量号 21H；功能号 (送 AH) 4CH)

2.设置中断向量 (21H；25H) [入口参数：DS=中断服务子程序入口段地址；DX=中断服务子程序入口相对地址；AL=中断类型号]

3.键盘输入一字符并显示 (21H；1) [出口参数：AL=输入字符的 ASCII 码]

4.字符串输入 (回车字符串输入结束符) (21H；0AH)

[入口参数：DS:DX 指向输入缓冲区；注：输入缓冲区第一个单元设置字符串最大输入长度，第二个单元为实际输入长度]

5.显示一字符 (21H；2) [入口参数：DL=待显示字符的 ASCII 码]

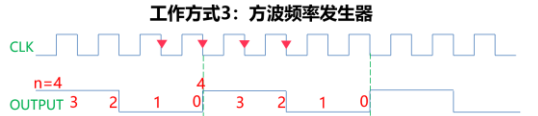
6.显示以“\$”结尾的字符串 (21H；9) [入口参数：DS:DX 指向字符串的首地址]

7.0-1FH, 80H-FOH 为 BIOS 中断向量号；20H-3FH 为 DOS；40H-7FH 用户备用

例：显示一个字符 (AH=2)	例：显示字符串 (AH=9)
MOV AH, 2	MOV DX, 'HELLO' ODH, OAH, '\$'
MOV DL, 'A'	；字符串以 '\$' 结尾
；待显示字符 ASCII 码置于 DL	MOV DX, OFFSET STRING
INT 21H	；DX 指向待显示字符串的首地址
	MOV AH, 9 ； INT 21H

例：设中断类型号为 8，则由此类型号中断向量表地址为：8×4=32=00100000B=20H ①由向量表可得对应 4 字节中断向量在表中位置：00020H_00021H_00022H_00023H ②假定中断类型 8 指定的中断向量为 CS=1000H，IP=0200H，即（00020H）=IP=00H，（00021H）=IP=02H，（00022H）=CS=00H，（00023H）=CS=10H，则由该中断向量形成的服务程序的入口地址将为 CS×16+IP=1000H×16+0200H=10200H。③CPU 一旦响应中断，将转向执行从地址 10200H 开始、类型号 8 的中断服务程序。

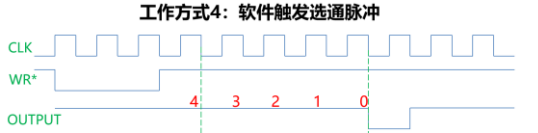
【例】方式3（方波频率发生）：（方式3 时序图1）



在GATE=1：输出频率为：fclk/n的方波。

- 写入控制字后，OUT端输出低电平作为起始电平，装入计数值n后，变为高电平。
 - n为偶数，每个时钟脉冲下降沿n值减1，至n/2后，电平变为低电平，并继续减1计数至0，然后改变OUT电平，重新装入n，开始计数。
 - n为奇数，输出高电平宽度为(n+1)/2,低电平宽度为(n-1)/2的方波。
- GATE=0：
- 停止计数，并强迫OUT输出高电平，在GATE变高后，重新将n装入，开始计数。

方式4（软件触发选通脉冲）：（方式4 时序图1 GATE 高电平+手动输入）



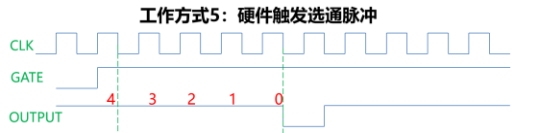
在GATE=1：

- 写入控制字后，OUT高电平，写入初始值后，计数器减1计数，OUT电平不变。
- 计数器减至0时，OUT输出一个脉冲周期的负脉冲，停止计数
- 只有输入新的计数值后，才能开始新的计数。

在GATE在计数中改变：

- 计数过程中，若GATE变低，停止计数，变高后，重新将n装入，开始计数。

方式5（硬件触发选通脉冲）：（方式5 时序图1 GATE 上升沿+自动置入）



在GATE产生一个上升沿：

- 写入控制字后，OUT高电平，写入初始值n后，须等待GATE上升沿才计数。
- 计数器减至0时，OUT输出一个脉冲周期的负脉冲。然后n值自动装入计数器，但要等GATE的上升沿来后才再次开始计数。
- 计数过程中，若GATE变低电平，不影响计数。如果是计数结束后，再一次产生GATE的上升沿，则再次计数。

初始化编程：在使用 8253 芯片时，首先要对其进行初始化编程，包括写入控制字和写入计数值两个步骤。

第一步，写入控制字。任一计数器的控制字都要从 8253 的控制口地址写入，控制那个计数器由控制字的 D7D6 位来决定。

第二步，写入计数初始值。计数初始值经由各计数器的端口地址写入。

若控制字里规定只写低八位，则写低八位，高 8 位自动置 0

若控制字里规定只写高八位，则写高八位，低 8 位自动置 0。

若是 16 位计数值，则分两次写入，先写低 8 位，再写高八位。

例 1：三个计数器端口地址为：40H，41H，42H，控制寄存器端口地址为：43H；时钟 1.19MHz。

计数器 0：电子时钟基准，工作于方式 3，计数器值预置为 0（即 65536），即输出的方波频率为：1.19/65536=18.21Hz，即每隔 55ms 产生一次中断请求。程序：

```
MOV AL, 36H
OUT 40H, AL; 先写低字节
MOV AL, 43H, AL; 写控制字
OUT 40H, AL; 再写高字节
```

（先后高后应置数 MOV AL, n；两次，此处特别两次都是 0，只写一次）

计数器 1：动态 RAM 定时刷新，工作于方式 2，计数器值预置为 18，即输出的频率为：1.19/18=66.1kHz，即每隔 15.1us 产生一次动态刷新请求信号。程序：

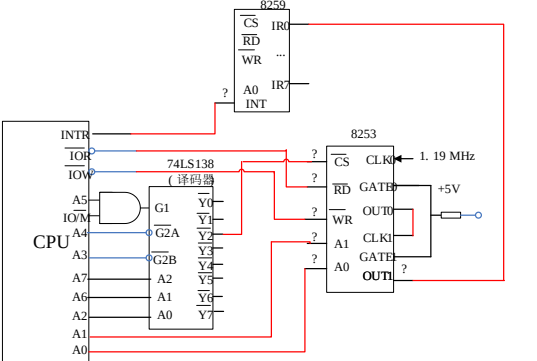
```
MOV AL, 54H
OUT 41H, AL; 预置计数值
MOV AL, 12H; 预置计数值
OUT 41H, AL; 写低字节
MOV AL, 43H, AL; 写控制字
OUT 43H, AL; 写控制字
```

计数器 2：扬声器音调，工作于方式 3，计数器值预置为 1331（533Hz），即输出的频率为：1.19/1331=894Hz。程序：

```
MOV AL, B6H
OUT 42H, AL; 先写低字节
MOV AL, 05H; 预置计数值
OUT 42H, AL; 再写高字节
```

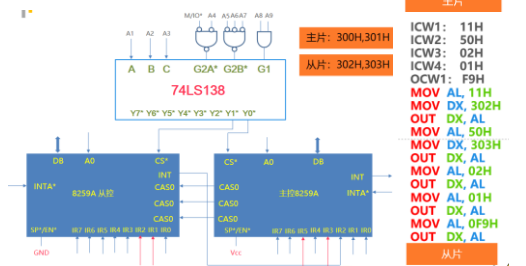
若计数值>65536，则通过两个计数器级联（前一个 OUT 接后一个 CLK）

例 2，图中设计采用 8253 计数器和 8259A 中断芯片，8253 的三个计数器端口地址为 60H-62H。控制寄存器端口地址为 63H，8253 每隔 10s 通过 out1 端口向 8259a 的 IR0 端口向 CPU 请求一次中断，请完成图中的硬件连线（？处），并写出 8253 的初始化程序。



此类问题分析如下：★8253 的 RD~WR 一般接 CPU 对应的 OR~OW；★8253 的 A0~A1 负责选择计数器(四个端口依次为 0001/0101)，接 CPU 对应的接口（一般是 A0/A1 但不一定）；★8253 的 CS 对应的数字 Yi 决定 CPU 的 A7/A6/A2 设置（本题为 010）；★74138 的 G1 对应的 A5 和 IO/M 设置为 1，G2A/G2B 对应的 A3/A4 设置为 0；★因此 8253 四个端口对应 CPU 的 A7-0 为 01100000B-01100011B 即为 60H-63H；★8253 的 OUT0 可以单独输出，计数值：n = 1.19MHz/(1/10)Hz=11900000>65536（n 最大值 65536），也可以接 CLK1，用 2 个计数器进行 2 次分频 n = n0 × n1；★8253 的 OUT1 接 8259 的 IR0(最高级中断)，8259 的 INT 接 CPU 的 INTR，即 8253 输出 OUT1 时，CPU 中断；

例 3，8259 级联中断的案例



可编程并行通信接口芯片 8255A：

该芯片有三组 8 位并行可编程端口 A、端口 B、端口 C，提供不同应用所要求的外部和设备进行连接的方法，它们既能够工作在同步传送方式，又能够工作在异步传送方式，既可以作为输入端口，也可以作为输出端口。（其中端口 C 可作为两个 4 位输入/输出端口，并可以配合 A、B 口做控制/状态信号）（寻址方式表1）

A1	A0	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	操作
0	0	0	1	0	读端口 A
0	1	0	1	0	读端口 B
1	0	0	1	0	读端口 C
0	0	1	0	0	写端口 A
0	1	1	0	0	写端口 B
1	0	1	0	0	写端口 C
1	1	1	0	0	写控制字

控制字：根据 D7 位的不同，控制字分为两类

方式选择控制字（D7=1）、C 端口置位复位控制字（D7=0）

方式选择控制字（D7=1）【★查询方式 0~2 前请务必先查询该表★】

D7	D6	D5	D4	D3	D2	D1	D0
1	A 口工作方式	A 口 I/O	C 口高 4 位	B 口工作方式	B 口 I/O	C 口低 4 位	

D6/D5：A 口工作方式 00 方式 0；01 方式 1；1X 方式 2

D2：B 口工作方式 0 方式 0；1 方式 1

I/O：输入输出方式 0 输出；1 输入

C 端口置位复位控制字：用于设置 C 口的某一位 D7=0

D7	D6	D5	D4	D3	D2	D1	D0
0	未用		位选择				1 置 0 复位

D3D2D1：000~111 分别选择 C 口位 0~7

方式 0：基本输入/输出工作方式（可用于无条件传送或查询式传送）

【★查询方式 0~2 前请务必先查询方式选择控制字表★】

特点：在该方式下 8255 提供了最基本的输入/输出工作方式,它没有“握手联络”信号，微处理器只要对端口进行最简单的读和写就能完成数据的输入和输出操作。

★1.在该方式下，8255A 可以定义为两个 8 位端口 AB，两个 4 位端口 C。★2.任何一个端口都可用作输入或输出。★3.由 A 口、B 口、C 口高 4 位与 C 口低 4 位 4 组合成 16 种不同的输入输出组态。★4.可用于同步传送和查询传送。在查询传送时，通常将 C 口的某些位规定为控制或状态信号端口，具体的功能可以由程序指定。

控制字格式如下：

D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	A	C 高 4 位	0	B	C 低 4 位

例：8255 的四个端口为：80H~83H。A 口工作方式 0，输出，B 口工作方式 0，输入。则指令 MOV AL, 1000010B；OUT 83H, AL；

方式 1：选通输入/输出工作方式(异步，可用于查询式传送或中断式传送)

特点：A 口、B 口作为 8 位数据端口,可以工作于选通输入/输出方式,当要在联络信号的控制下才能完成 I/O 操作:C 口的 6 根 I/O 线作为 A 口、B 口的联络信号:C 口余下的两根 I/O 线仍能用来输入输出数据。

端口 A 和 B 均为输入方式控制字格式：

D7	D6	D5	D4	D3	D2	D1	D0
1	0	1	1	PC6 PC7：I/O		1	1 X

端口 A 和 B 均为输出方式控制字格式：

D7	D6	D5	D4	D3	D2	D1	D0
1	0	1	0	PC4 PC5：I/O		1	0 X

混合输入与输出控制字格式：

D7	D6	D5	D4	D3	D2	D1	D0
1	0	1	↓	↓	↓	1	↓ X

D4：A 口方式 1：输入；0 输出

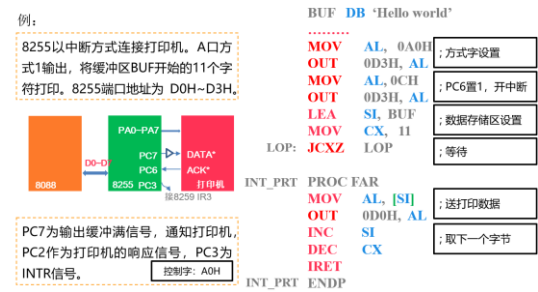
D3：PC6、PC7 1：输入；0 输出

D1：B 口方式 1：输入；0 输出

例：8255 的四个端口为：80H~83H。要对 PC7 置 1，对 PC3 置 0。则指令：

MOV AL, 0FH；OUT 83H, AL；PC7 置 1：00001111B

MOV AL, 06H；OUT 83H, AL；PC3 置 0：00000110B



方式 2：选通双向传输(异步,仅适用于端口 A)

特点:双向方式中 A 端口在某一时刻工作在输入方式,在另一时刻又可以工作在输出方式,需要有两组 6 位联络信号,分别与输入和输出设备进行联络,联络信号由 C 端口中的 5 个數位提供,其中输入和输出的中断请求信号合并为一个;联络信号由 C 端口提供,C 端口只有 8 位,只能为一个端口提供联络在方式 2 下工作,因此在 8255A 中只有 A 组控制的 A 端口能够工作在双向方式,控制字格式如下↓

D7	D6	D5	D4	D3	D2	D1	D0
1	1	X	X	X	↓	↓	↓

D2：B 口工作方式 1：方式 1；0：方式 0

D1：PA0 至 PA7 方式 1：输入；0：输出

D0：PC0 至 PC2 方式 1：输入；0：输出

时序图：太复杂，编者看不懂，故不放了

例题（2012 年第五题）下图为 8086 系统的一个接口电路，有一片 8259，一片 8255A 及 8 个发光二极管，通过按钮开关可以产生中断请求信号。要求通过 8259 产生中断请求，在中断服务程序中通过 8255A 的 A 口输出控制 8 个发光二极管轮流点亮，8259 的口地址为 40H、41H，中断类型为 80H~87H，工作方式为：边沿触发，一般全嵌套，一般结束中断，非缓冲方式；8255A 按基本输入输出方式工作，端口地址为 60H~63H。

- 1) 请写出 8259 的初始化程序，并设置中断屏蔽字 OCMW1；
- 2) 写出 8255A 的初始化程序，给 8255A 的 A 口设置初值，将初值保存到“INIT”单元；
- 3) 请编程设置中断向量，假设中断服务程序入口处的标号为“LIGHT_INT”；
- 4) 请写出对应的中断服务程序（包括开中断、8255A 的 A 口输出控制 8 个发光二极管轮流点亮、给 8259 发 EOI 命令、中断返回）。

1) 8259 初始化（4 分）

```
MOV AL,00010011B; 写 ICW1
OUT 40H, AL
MOV AL, 80H; 中断类型号
OUT 41H, AL; 写 ICW2
MOV AL,00000001B; 写 ICW4
OUT 41H, AL
MOV AL, 0; 写 OCMW1 屏蔽字，允许中断
OUT 41H, AL
或 IN AL, 41H
AND AL, 11110111B
OUT 41H, AL; 写 OCMW1 屏蔽字，允许 IR3 中断
```

2) 8255A 初始化（4 分）

```
MOV AL, 80H; 写控制字
OUT 63H, AL
MOV AL, 01H
OUT 60H, AL; 给 8255A 的 A 口设置初值，D1 亮，其它不亮。
MOV INIT, AL
```

3) 设置中断向量（4 分）

```
MOV AX, SEG LIGHT_INT
MOV DS, AX
MOV DX, OFFSET LIGHT_INT
MOV AL, 83H ;对应 IR3
MOV AH, 25H
INT 21H
```

4) 中断服务程序（4 分）

```
LIGHT_INT: STI ; 开中断
MOV AL, INIT
ROL AL, 1; 控制发光二极管轮流点亮
OUT 60H, AL; 从 A 口送出
MOV INIT, AL
MOV AL, 20H; 设置 8259OCW2，EOI=1，结束 8259 中断
OUT 40H, AL; 发出 OCMW2
IRET ; 中断返回
```

（每按一次进入中断程序）

